Миша Филитов

АКОС 7
Системные вызовы, файлы, дескрипторы

НИУ ВШЭ ФКН ПМИ 2026

Задачи ОС

- Распределять процессорное время и память между программами
- Изолировать процессы и проверяет права доступа
- Предоставляет файлы, процессы и сокеты
- Управлять устройствами через драйверы

Системный вызов

- Записать/прочитать что-то из файла
- Получить немного памяти
- Создать новый процесс
- Повзаимодействовать с внешним устройством

Как сделать системный вызов?

- RAX - номер системного вызова
- RDI, RSI, RDX, R10, R8, R9 - аргументы
- syscall
- Возвращаемое значение - RAX. Если RAX < 0 - случилась ошибка

- Справка - man 2 syscall
- В отличие от system V ABI четвертый аргумент не RCX
- В си используются обертки над сисколами, они используют стандартное соглашение о вызовах. Ошибка выставляется в специальную переменную errno.

```c
#include <unistd.h>

ssize_t read(int fd, void buf[.count], size_t count);
```

man 2 read

- **fd** - файловый дескриптор, из которого нужно читать
- **buf** - буффер, куда нужно записать прочитанное
- **count** - прочитать не больше, чем это число
- **return**
  0: EOF (end of file)
  >0: число реально прочитанных байт
  <0: ошибка. Номер ошибки в errno

```c
#include <unistd.h>

ssize_t write(int fd, const void buf[.count], size_t count);
```

man 2 write

- **fd** - файловый дескриптор, в который нужно писать
- **buf** - буффер, откуда нужно читать
- **count** - записать не больше, чем это число
- **return**
  >0: число реально записанных байт
  <0: ошибка. Номер ошибки в errno

ВАЖНО!

- write/ read не обязаны записывать/считывать столько байт, сколько указано. Они могут записать/прочитать меньше. *Для* корректной работы нужен цикл.

errno

- Доступен через **<errno.h>**
- Системный вызов возвращает отрицательное число и выставляет **errno**
- **strerror(errno)** - получить текст ошибки
- **perror("msg")** - напечатать текст ошибки в stderr
- Следующий системный вызов может перетереть **errno**
- Если нужно его сохранить, то можно сделать **int error = errno;**

код

Вывод

- Системные вызовы делать дорого!
- Вся наша жизнь - попытки уменьшить число системных вызовов

```c
#include <fcntl.h>

int open(const char *pathname, int flags, ... /* mode_t mode */ );
```

man 2 open

- **pathname** - путь к файлу
- **flags** - режим открытия и дополнительные флаги
  - O_RDONLY - открыть для чтения
  - O_WRONLY - открыть для записи
  - O_RDWR - открыть для чтения и записи
  - O_CREAT - создать файл, если он не существует
  - O_TRUNC - очистить содержимое существующего файла
  - O_APPEND - записывать в конец файла
- **mode** - права доступа при создании нового файла
- **return**:
  - >= 0 - файловый дескриптор (обычно первый свободный)
  - < 0 - ошибка. Значение в errno
- **Note:** после open надо делать **close(fd)**

```c
#include <unistd.h>

off_t lseek(int fd, off_t offset, int whence);
```

man 2 lseek

- **fd** - файловый дескриптор
- **offset** - смещение в байтах
- **whence** - относительно чего вычисляется новая позиция

  SEEK_SET - от начала файла

  SEEK_CUR - от текущей позиции

  SEEK_END - от конца файла
- Поддерживается не для всех объектов (для сокетов или пайпов)
- **lseek(fd, 0, SEEK_CUR)** - узнать текущую позицию без её изменения

Файловый дескриптор

- неотрицательное целое число, через которое процесс обращается к открытому файлу или другому объекту

- У каждого процесса есть таблица файловых дескрипторов
- **fd** - индекс в этой таблице, по которому ядро находит открытое файловое описание
- При **open()** ядро обычно возвращает минимальный свободный **fd**
- При **close()** дескриптор освобождается и может быть использован повторно
- Несколько **fd** могут ссылаться на одно открытое файловое описание
- При старте процесса есть 3 fd: 0 - stdin, 1 - stdout, 2 - stderr

Файловый дескриптор

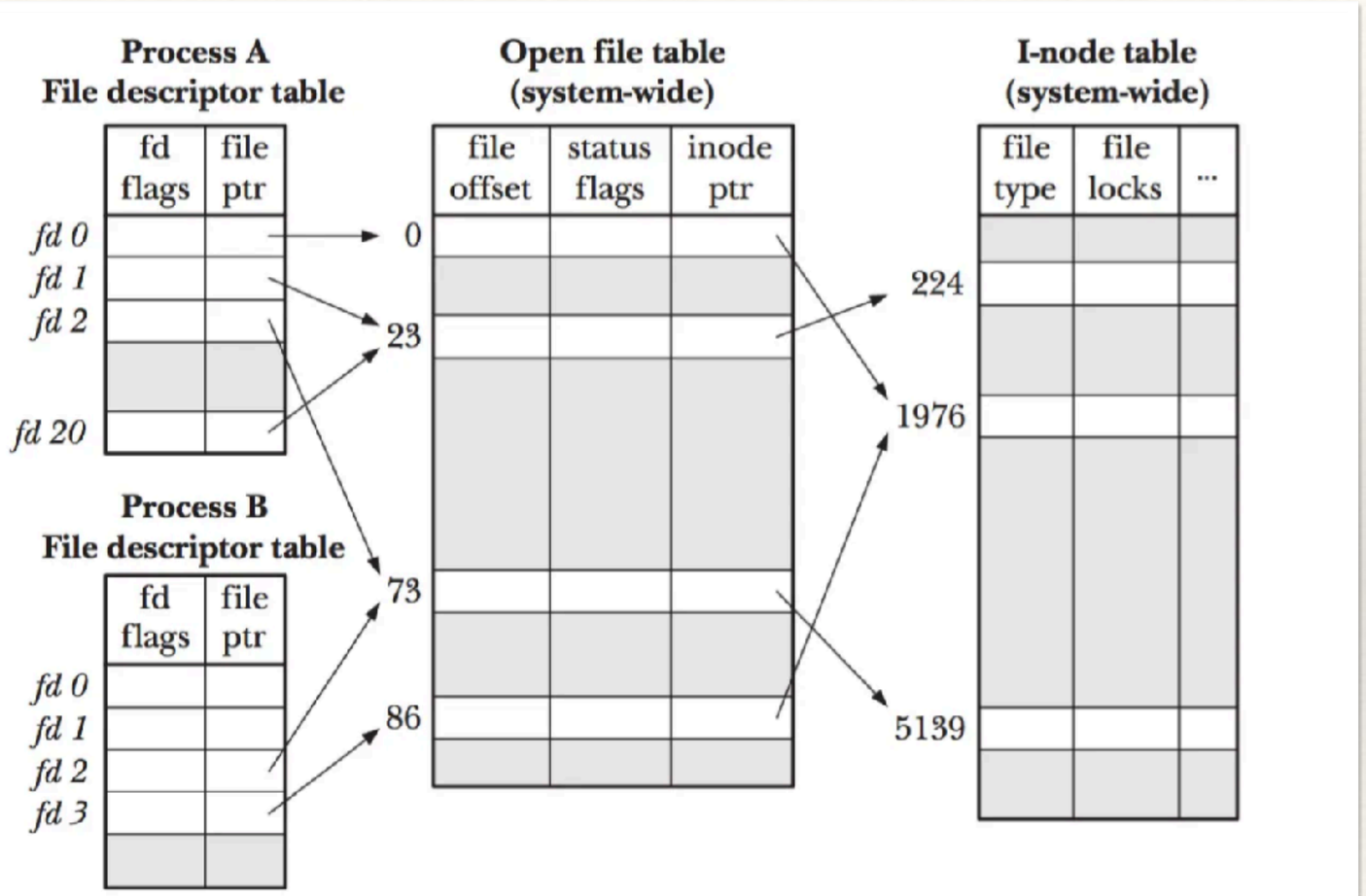


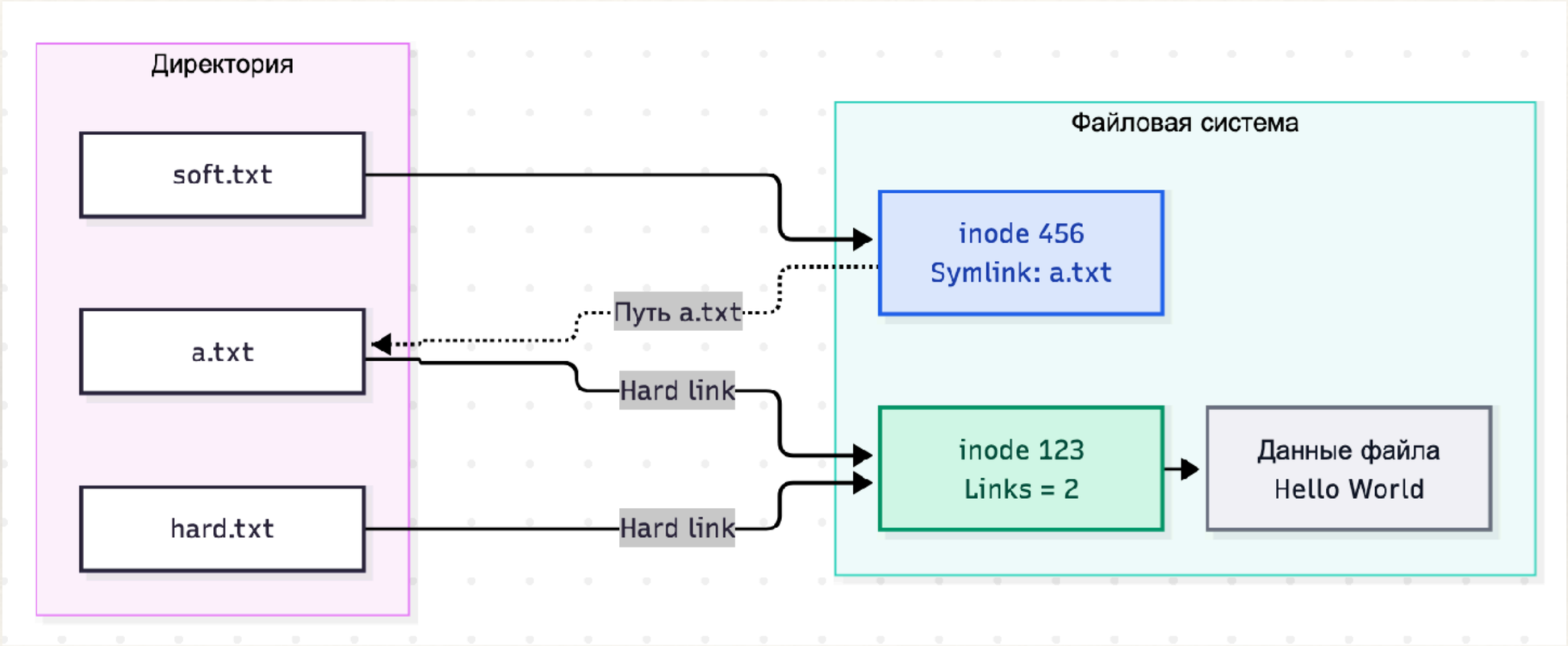
Figure 5-2: Relationship between file descriptors, open file descriptions, and i-nodes

Ссылки

- hard link - дополнительное имя для уже существующего inode
- symlink - отдельный файл, содержащий путь к другому файлу (ярлык)

	Hard link	Symlink
Создание	ln old new	ln -s old new
Inode	Тот же	Разные
Удаление old	Продолжает работать	Ссылка битая
Между файловыми	Нет	Да
Ссылка на директорию	Нет	Да

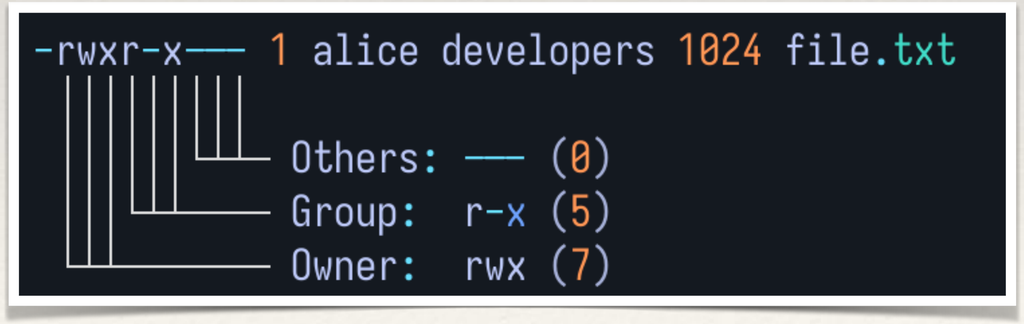
Ссылки


Директория
soft.txt
a.txt
hard.txt
Файловая система
inode 456
Symlink: a.txt
Путь a.txt
Hard link
Hard link
inode 123
Links = 2
Данные файла
Hello World

Права

- Является ли текущий процесс владельцем файла?
  да - Owner права
  нет - дальше
- Входит ли процесс в группу файла?
  да - Group права
  нет - дальше
- Other

Бит	Для файла	Для директории
r	Чтение содержимого	Просмотр списка имён
w	Изменение содержимого	Создание/удаление записей (с x)
x	Выполнение программы	Доступ к объектам по имени, проход



<dirent.h>

- **opendir(path)** - открыть директорию, возвращает **DIR***
- **readdir(dir)** - получить следующую запись директории
- **closedir(dir)** - закрыть директорию
- **rewinddir(dir)** - вернуться к началу директории